

sLLM Fine-Tuning 실습 서브노트_1일차

HR 규정 안내 챗봇 LoRA SFT (Qwen2.5-1.5B-Instruct)

판교캠퍼스 8반 최규원

1일차 핵심 학습 내용과 느낀점

이번 수업에서는 범용 LLM을 기업 내부 업무에 그대로 적용하기 어려운 이유와, Knowledge Asset을 활용해 특정 도메인에 맞는 sLLM 서비스를 구성하는 방법을 학습했습니다. 실습은 Qwen2.5-1.5B-Instruct를 생성 모델로 사용하고, BGE-M3와 FAISS를 연결한 Mini Knowledge Base를 구성한 뒤 PEFT의 대표 기법인 LoRA를 적용하기 전과 후의 파라미터 구조를 비교하는 순서로 진행했습니다.

수업 전에는 회사 문서를 모델이 활용하게 만드는 방법을 전부 Fine-Tuning으로 뭉뚱그려 생각했는데, 실습 후에는 지식을 모델 외부에 두고 질문할 때마다 검색하는 방법(RAG)과, 질문-답변 예시로 모델의 응답 방식·행동 자체를 학습시키는 방법(SFT)이 서로 다른 문제를 해결한다는 걸 이해하게 됐습니다.

- RAG·Semantic Search는 최신 지식과 근거 문서를 모델의 입력에 제공합니다.
- SFT는 모델이 어떤 형식과 태도로 답해야 하는지를 반복 예시로 학습시킵니다.
- LoRA는 Base Model 전체를 다시 학습하지 않고 작은 Adapter만 학습해 자원 부담을 낮춥니다.

결국 기업용 AI에서는 가장 큰 모델을 쓰는 것보다, 어떤 Knowledge Asset을 어떤 구조로 연결하고 어떤 데이터로 행동을 학습시킬지가 더 중요한 설계 문제라고 느꼈습니다.

Chapter 1. 이해관계자 가치와 Pain Point

1. 범용 LLM을 사내 업무에 적용할 때의 한계

범용 LLM은 인터넷 전체를 학습해서 언어 능력 자체는 뛰어나지만, Human AI Corporation의 사내 규정이나 업무 프로세스, 전문 용어는 애초에 학습 데이터에 존재하지 않기 때문에 알지 못합니다. “재택근무 규정”이나 “LYNX 출장 정산 기준” 같은 질문을 그대로 던지면 일반론적이거나 부정확한 답변만 돌아오는 이유가 여기에 있습니다.

이 상태에서 범용 모델을 그대로 사내 규정 안내에 쓰면 다음과 같은 문제가 생길 수 있습니다.

- **지식 공백**: 모델이 회사 고유 규정과 내부 용어를 알지 못합니다.
- **Hallucination**: 일반적인 기업 관행을 실제 사내 규정처럼 답할 수 있습니다.
- **최신성 부족**: 규정이 변경돼도 기존 모델의 가중치에는 자동으로 반영되지 않습니다.
- **보안 위험**: 사내 규정을 상용 LLM API에 그대로 입력하면 데이터가 외부 서버로 전송됩니다. VPC나 전용 인스턴스로 사내망에 배치해도 학습 데이터 활용 정책 자체는 여전히 벤더에 달려 있어서, 계약서로 “학습에 사용하지 않는다”고 명시해도 기술적으로 완전히 검증하기는 어렵습니다.

2. 이해관계자별 가치

이해관계자	겪는 불편함	sLLM 도입 시 얻는 가치
일반 직원	규정을 찾으려면 인사팀에 문의하거나 매뉴얼을 직접 검색해야 함	챗봇에 바로 질문에 즉시 답변 획득
인사팀	반복적인 단순 문의 응대에 시간 소모	반복 문의 자동 처리로 업무 부담 경감
회사(보안 담당)	사내 규정을 상용 LLM API에 입력하면 외부 서버로 데이터가 전송될 위험	완전 내부 운용이 가능해 데이터가 외부로 나가지 않음

3. 왜 sLLM인가

sLLM의 가치는 단순히 파라미터 수가 작다는 데 있지 않습니다. 데이터 통제를 “계약”이 아니라 “아키텍처”로 보장할 수 있고, 범용 모델보다 좁지만 우리 회사 도메인에 맞게 깊게 특화시킬 수 있다는 데 의미가 있습니다. 이번 과제의 HR 규정 안내처럼 업무 범위가 분명한 경우에는, 모든 분야를 아는 거대한 모델보다 회사 규정을 근거로 일관되게 답하고 모르는 내용은 정직하게 거절하는 모델이 실무적으로 더 유용할 수 있습니다.

Chapter 2. Knowledge Asset과 sLLM 활용

1. Knowledge Asset의 의미

Knowledge Asset은 기업이 축적한 사내 규정, 업무 매뉴얼, 기술 문서, FAQ, 전문 용어, 승인 절차 같은 고유한 지식 자산을 의미합니다. 범용 모델이 이런 정보를 전부 기억하게 만드는 대신, 문서를 검색 가능한 형태로 관리하고 필요한 순간에 관련 부분만 모델에 전달할 수 있습니다.

다. 즉 지식을 모델 안에 넣는 대신, 모델 밖에 두고 그때그때 꺼내 쓰는 방식이며, 뒤에서 다룰 파라미터 학습(SFT)과는 완전히 다른 층위의 해결책입니다.

1일차 실습에서는 다음 6개의 Knowledge Document를 사용했습니다.

- **HR-EDU-001**: AI 교육비 지원 한도
- **HR-EDU-002**: AI 교육비 신청 기한
- **HR-EDU-003**: AI 교육비 승인 절차
- **HR-EDU-004**: AI 교육비 지원 제외 조건
- **HR-LEAVE-001**: 리프레시 휴가
- **HR-WORK-001**: 재택근무

2. BGE-M3, FAISS, Qwen의 역할

구성 요소	역할
BGE-M3	문서와 질문의 의미를 벡터(Embedding)로 변환
FAISS IndexFlatIP	정규화된 벡터를 저장하고 질문과 가까운 문서를 검색
Qwen2.5-1.5B-Instruct	검색된 Knowledge를 Context로 받아 자연어 답변 생성

실습은 `knowledge_asset_demo_faiss_bge_m3.ipynb` 에서 진행했습니다. 문서 1건을 Chunk 1개로 정의하고, title과 content를 합쳐 BGE-M3로 임베딩해 FAISS `IndexFlatIP` 에 저장했습니다(`faiss_index.ntotal == 6`). 질문이 들어오면 질문도 동일한 방법으로 임베딩한 뒤 FAISS에서 유사도가 높은 Top-2 문서를 검색하고, 검색된 문서만 Context로 구성해 Qwen에 전달합니다. 세 구성 요소의 역할이 분리되어 있다는 점이 중요한데, BGE-M3는 답을 생성하지 않고 의미를 벡터로 바꾸기만 하고, FAISS는 문서를 읽고 판단하지 않고 벡터 간 유사도만 계산하며, 최종 답변 생성은 Qwen이 전담합니다.

3. 실습 결과에서 확인한 점

질문	검색 결과	Qwen 답변에서 확인한 내용
AI 교육비는 연간 얼마까지 지원받을 수 있나요?	HR-EDU-001 (score 0.7812), HR-EDU-004 (score 0.6470)	"AI 교육비는 연간 120만원까지 지원받을 수 있습니다"라고 정확히 답변
해외 AI 컨퍼런스 항공료를 전액 지원하나요?	관련성이 낮은 Chunk만 검색(score 0.56, 0.45)	"현재 Knowledge Base에서 해당 내용을 확인할 수 없습니다"라고 정직하게 답변

Base Qwen은 회사의 교육비 한도를 원래 알고 있던 게 아닙니다. 검색된 Knowledge를 읽고 답변을 생성한 것이며, 모델 가중치를 전혀 바꾸지 않고도 외부 Knowledge를 연결해 특정 업무 질문에 대응할 수 있음을 확인했습니다.

```
knowledge_asset_demo_faiss_bge_m3.ipynb

In [25]:
search_results = search_knowledge(question, top_k=2)
answer = generate_answer(question=question, retrieved_documents=search_results)

[Question]
AI 교육비는 연간 얼마까지 지원받을 수 있나요?

[Retrieved Knowledge]
- HR-EDU-001 | AI 교육비 지원 한도 | score=0.7812
- HR-EDU-004 | AI 교육비 지원 제외 조건 | score=0.6470

[Qwen Answer]
AI 교육비는 연간 120만원까지 지원받을 수 있습니다.

In [26]:
unknown_question = "Human AI Corporation은 해외 AI 컨퍼런스 참석 시 항공료를 전액 지원하나요?"

[Unknown Question]
Human AI Corporation은 해외 AI 컨퍼런스 참석 시 항공료를 전액 지원하나요?

[Retrieved Knowledge]
- HR-EDU-001 | AI 교육비 지원 한도 | score=0.5647
- HR-EDU-003 | AI 교육비 승인 절차 | score=0.4470

[Qwen Answer]
현재 Knowledge Base에서 해당 내용을 확인할 수 없습니다.
```

```
knowledge_asset_demo_faiss_bge_m3.ipynb - extra questions

In [27]: # 추가 질문 5개로 검색 다양성 확인

=====

[Question 1]
AI 교육비 신청은 교육 시작 며칠 전까지 해야 하나요?

[Retrieved]
- HR-EDU-002 | AI 교육비 신청 기한 | 0.8096
- HR-EDU-003 | AI 교육비 승인 절차 | 0.6302

[Answer]
AI 교육비 신청은 교육 시작 일 기준 최소 14일 전에 완료해야 합니다.

=====

[Question 4]
재택근무는 일주일에 몇 번 가능한가요?

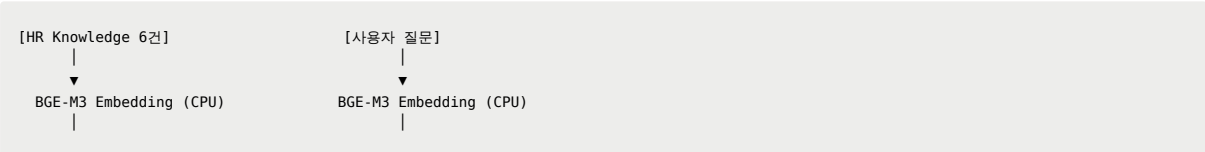
[Retrieved]
- HR-WORK-001 | 재택근무 | 0.7673
- HR-LEAVE-001 | 리프레시 휴가 | 0.4619

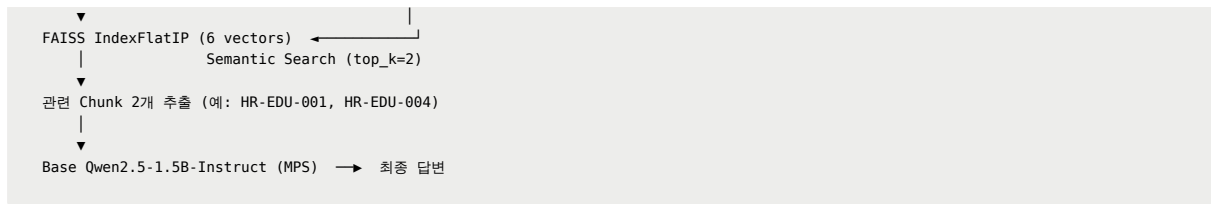
[Answer]
재택근무는 주 최대 2회까지 가능합니다.

=====
```

Chapter 3. Mini Knowledge Base 구성도

1. 구성 (Mac, Apple Silicon 환경 기준)





왼쪽 경로(문서 → 임베딩 → FAISS 저장)는 사전에 1회만 수행하고, 오른쪽 경로(질문 → 임베딩 → 검색)는 질의마다 수행됩니다. Mac(Apple Silicon) 환경이라 `faiss-cpu` 패키지로 실제 FAISS `IndexFlatIP`를 사용했고(NumPy 풀백이 아닙니다), Qwen 생성만 `mps` 장치를, BGE-M3 임베딩은 `cpu`를 쓰도록 장치를 분리했습니다.

2. 구조의 장점과 한계

이 구조의 장점은 모델을 다시 학습하지 않아도 문서 변경을 빠르게 반영할 수 있고, 검색된 문서를 답변 근거로 제시할 수 있다는 점입니다. 반면 한계도 실습 중에 직접 확인했는데, FAISS의 최근접 검색(`IndexFlatIP`)은 “관련 문서가 없다”는 판단을 하지 않고 저장된 벡터 중 가장 가까운 것을 항상 반환합니다. 실제로 “수료율이 70%라면 교육비 지원을 받을 수 있나요?”라는 질문에서, 검색 자체는 정확한 근거 Chunk(`HR-EDU-004`: “수료율 80% 미만이면 제외”)를 찾아냈는데도 Qwen이 “네, 지원받을 수 있습니다”라고 오답한 사례가 있었습니다 (70% < 80%이므로 실제로는 제외 대상). 즉 **관련 Chunk가 검색됐다는 사실과, 그 내용을 LLM이 올바르게 추론한다는 것은 별개의 문제**라는 걸 확인했습니다. 검색 단계뿐 아니라 생성 단계에도 별도의 검증이 필요하다는 뜻이고, RAG만으로는 부족한 부분을 SFT로 조건문 해석을 정확히 가르쳐 보완할 필요가 있다고 판단했습니다.

Chapter 4. PEFT(LoRA) 적용 전·후 Parameter 비교

1. Full Fine-Tuning과 PEFT

Full Fine-Tuning은 Base Model의 전체 가중치를 학습합니다. 자원 비용이 크고, 기존에 갖고 있던 일반 능력이 손상되는 Catastrophic Forgetting 위험도 있습니다. PEFT는 원본 가중치를 전부 동결(Frozen)하고 소수의 추가 파라미터만 학습해 이 부담을 줄이는 방법이고, LoRA는 그 대표 기법입니다. “15억 개 파라미터 중 일부만 골라서 학습하는 방법”으로 이해하면 정확하지 않습니다. 실제로는 기존 파라미터를 전부 그대로 둔 채, 선택한 레이어(`target_modules`) 옆에 작은 저랭크 행렬(A, B)을 새로 추가하고 그 행렬만 학습합니다.

```

기존 출력:      W × x
LoRA 적용 출력: W × x + scale × B(A(x))

W   : Base Model의 동결된 가중치
A, B : 새로 추가되어 학습되는 저랭크 행렬
  
```

숫자로 감을 잡아보면, Linear Layer 하나가 1536×1536 크기일 때 원본 전체를 학습하면 2,359,296개 파라미터가 필요하지만, LoRA(r=8)로 대신 표현하면 (8×1536)+(1536×8) = 24,576개, 즉 이 레이어 하나만 봐도 학습 대상이 약 1% 수준으로 줄어듭니다. 이게 가능한 이유는 원본 파라미터(W)가 이미 완성도 높은 상태라서, 새 도메인에 맞추기 위해 실제로 필요한 변화가 원본 전체 복잡도만큼 크지 않다는 저랭크 가설(Low Intrinsic Rank Hypothesis)이 실험적으로 검증되었기 때문입니다.

2. 실습 설정

```

lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    lora_dropout=0.05,
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj"
    ],
    task_type="CAUSAL_LM",
)
  
```

교재 예시(`q_proj`, `v_proj` 2개, r=8)보다 Rank를 2배(r=16)로, 대상 레이어도 Attention 4개(`q_proj`, `k_proj`, `v_proj`, `o_proj`) + MLP 3개(`gate_proj`, `up_proj`, `down_proj`)까지 총 7개로 넓혀서 실습했습니다. `target_modules`는 Adapter를 설치할 레이어의 위치이고, 여기 지정한 이름은 한 곳에만 붙는 게 아니라 Qwen의 28개 Transformer Layer 각각에 반복 적용됩니다.

3. 적용 전·후 비교

`scripts/train_lora.py` 실행 시 실제로 출력된 값은 다음과 같습니다.

```
trainable params: 18,464,768 || all params: 1,562,179,072 || trainable%: 1.1820
```

구분	전체 Parameter	Trainable Parameter	Trainable 비율
LoRA 적용 전 (Base)	1,543,714,304	1,543,714,304	100.0000%
LoRA 적용 후 (r=16, 7개 모듈)	1,562,179,072	18,464,768	1.1820%

LoRA 적용 후 전체 Parameter는 오히려 18,464,768개 늘어납니다. 이 증가분이 새로 추가된 Adapter Parameter이고, 기존 Base Parameter(1,543,714,304개)는 학습 대상에서 제외됩니다. 즉 “전체 파라미터 수가 줄어든다”가 아니라 “실제로 학습해야 하는 파라미터 비율이 크게 줄어든다”가 정확한 설명입니다.

Qwen2.5-1.5B의 구조(hidden size 1536, MLP intermediate size 8960, Attention head 12개 중 Key/Value head는 2개라 `k_proj` / `v_proj`의 출력 차원이 256)를 바탕으로 레이어 1개에서 늘어나는 Parameter를 직접 계산해보면 위 실행 결과와 정확히 일치합니다.

```

scripts/train_lora.py

(sllm) $ python scripts/train_lora.py

[Device] Selected device: mps (Apple Silicon GPU)
[Model] Loading base model: Qwen/Qwen2.5-1.5B-Instruct
[LoRA] r=16, alpha=32, dropout=0.05
[LoRA] target_modules=[gate_proj, up_proj, q_proj, v_proj, down_proj, k_proj, o_proj]

trainable params: 18,464,768 || all params: 1,562,179,072 || trainable%: 1.1820

[Training] epochs=2.0 max_length=384 lr=1e-04 grad_accum=8
[Training] global_step=146 final_train_loss=0.6735 best_metric=0.2170
[Training] best_checkpoint=models/hr-qwen-lora/checkpoint-146

```

대상	Shape (A)	Shape (B)	Parameter 수
q_proj	[16, 1536]	[1536, 16]	49,152
k_proj	[16, 1536]	[256, 16]	28,672
v_proj	[16, 1536]	[256, 16]	28,672
o_proj	[16, 1536]	[1536, 16]	49,152
gate_proj	[16, 1536]	[8960, 16]	167,936
up_proj	[16, 1536]	[8960, 16]	167,936
down_proj	[16, 8960]	[1536, 16]	167,936
레이어당 합계			659,456

$659,456 \times 28\text{개 레이어} = 18,464,768$ — `train_lora.py`가 출력한 실제 값과 정확히 일치합니다. `target_modules`를 교재 예시보다 넓게(7개) 잡고 `r`도 2배(16)로 키웠기 때문에, 교재 예시의 trainable%(0.32%)보다 약 3.7배 높은 1.18%가 나왔다는 것을 계산 구조로 직접 확인할 수 있었습니다. MLP 레이어(`gate_proj`, `up_proj`, `down_proj`)의 기여도가 Attention 레이어보다 훨씬 크다는 점도 이 표에서 드러납니다(레이어당 167,936개 $\times$ 3 vs Attention 4개 합쳐도 155,648개).

한 가지 더 확인한 점은, LoRA를 붙인 직후 학습을 시작하지 않았는데도 `print_trainable_parameters()`가 이미 1.1820%를 표시한다는 것입니다. 이는 `target_modules`와 `r`을 지정하는 순간 A-B 행렬의 Shape와 개수가 즉시 확정되기 때문이며(구조 결정), 그 안에 실제로 의미 있는 값이 채워지는 것은 이후 forward-loss-backward-optimizer step을 반복하는 학습 과정(값 결정)입니다. 구조가 만들어지는 시점과 값이 학습되는 시점이 서로 다른 단계라는 걸 이 지점에서 체감했습니다.

Chapter 5. SFT 적용 방향과 다음 실습의 연결

1. SFT가 학습시키는 것

SFT는 (질문, 정답) 쌍을 모델에 보여주고 “이렇게 답하라”는 형식·태도를 가르치는 지도 미세조정 방식입니다. 회사 문서를 그대로 넣는 게 아니라, 실제 서비스 대화 형식인 `system` / `user` / `assistant` 메시지로 구성합니다.

```

{
  "messages": [
    {"role": "system", "content": "당신은 Human AI Corporation의 인사 규정 안내 도우미입니다."},
    {"role": "user", "content": "재택근무 규정을 설명해 주세요."},
    {"role": "assistant", "content": "재택근무는 회사가 승인한 직무에 한하여..."}
  ]
}

```

SFT 데이터는 지식 내용뿐 아니라 답변 형식, 어투, 모르는 질문을 안전하게 거절하는 행동까지 반복해서 보여줍니다. Human AI Corporation의 Knowledge Asset(HR 매뉴얼)과 업무 요구사항을 이런 형식의 학습 데이터로 구성하면, Base Qwen을 HR 규정에 특화된 모델로 만들 수 있습니다.

2. RAG와 SFT 비교

비교 항목	RAG · Semantic Search	SFT · LoRA
정보/학습 결과의 위치	모델 외부의 문서·Vector DB	학습된 변화량이 LoRA Adapter Parameter에 반영
적용 시점	질문마다 관련 문서 검색	학습 시 질문·모범답변 반복
주요 역할	최신 정보, 근거 추적, 문서 교체 용이	답변 형식·행동·업무 규범을 모델에 학습
갱신 방법	문서와 Index 갱신	데이터 수정 후 Adapter 재학습
주요 위험	검색 실패, 무관한 Context 주입 (Chapter 3-2 참고)	과적합, 학습되지 않은 최신 규정, 데이터 편향

두 방법은 대체 관계가 아닙니다. SFT로 회사가 원하는 응답 방식과 안전한 행동을 학습시키고, RAG로 최신 규정과 근거 문서를 제공하는 조합이 실무적으로 적절하다고 판단했습니다.

3. 다음 실습에서 확인할 내용

배포 데이터는 `dataset/training/hr_sft_train.jsonl` (Train, 582건), `hr_sft_validation.jsonl` (Validation, 97건), `dataset/evaluation/hr_eval.jsonl` (Evaluation, 20건)로 구성되어 있습니다. 다음 실습에서는 `train_lora.py` 로 LoRA Adapter를 학습하고 `evaluate_model.py` 로 Base 모델과 Fine-tuned 모델의 응답을 비교합니다.

이때 확인해야 할 핵심은 단순히 학습 Loss가 줄었는지가 아니라, 회사 고유 코드와 필수 키워드를 정확히 반영하는지(Keyword Score), 참조 답변과 핵심 내용이 얼마나 일치하는지(Token F1), 규정에 없는 질문에 내용을 지어내지 않고 안전하게 거절하는지(Hallucination Rate) 까지 함께 평가하는 것입니다.

```

scripts/evaluate_model.py

(sllm) $ python scripts/evaluate_model.py

[Evaluation Summary]
Evaluation items : 20
Known policies : 15
Unknown policies : 5
Win margin : 0.0100

Base wins : 1
Fine-tuned wins : 18
Ties : 1

[Base Model]
Keyword score : 0.065
Token F1 : 0.0964
Total score : 0.0862
Hallucination : 0/5

[Fine-tuned Model]
Keyword score : 0.2
Token F1 : 0.2198
Total score : 0.285
Hallucination : 0/5

[Improvement]
Keyword : +0.1350
Token F1 : +0.1234
Total : +0.1989
Hallucination : +0.0000

Detailed results : outputs/evaluation_results.jsonl
Summary : outputs/evaluation_summary.json

```

Chapter 6. 트러블슈팅, 한계 및 최종 정리

1. Jupyter 커널이 프로젝트 가상환경과 분리되는 문제

- **현상:** VS Code에서 `peft_lora_parameter_compare_cache.ipynb` 의 `from transformers import ...` 셀 실행 시 `ModuleNotFoundError: No module named 'transformers'` 발생
- **원인:** 노트북의 Jupyter 커널이 `sllm` 가상환경이 아닌 시스템 Python(3.14)에 연결되어 있었음
- **해결:** `sllm/bin/python3 -m ipykernel install --user --name=sllm --display-name="Python (sllm)"` 로 `venv`를 커널로 등록한 뒤, VS Code 커널 선택 메뉴에서 "Python (sllm)"으로 전환
- **배운 점:** Jupyter 커널과 Python 가상환경은 별개로 관리된다는 것을 체감했습니다. 노트북을 열 때는 항상 커널이 어느 인터프리터를 가리키는지부터 확인해야 합니다.

2. 터미널 로그 스크롤 유실 문제

- **현상:** `train_lora.py` 실행 중 터미널 로그의 앞부분(초기 loss 등)이 스크롤 밖으로 밀려 확인 불가
- **원인:** VS Code 통합 터미널의 기본 `scrollback(1000줄)`이 전체 학습 로그 길이보다 짧음
- **해결:** `python scripts/train_lora.py 2>&1 | tee outputs/train_log.txt` 로 로그를 파일에도 저장하고, `terminal.integrated.scrollback` 설정 값을 상향 조정
- **배운 점:** 장시간 걸리는 학습 스크립트는 콘솔 출력에만 의존하지 말고, 처음부터 파일로 함께 남기는 습관을 들이는 게 낫다고 느꼈습니다.

3. 검색은 성공해도 추론은 실패할 수 있다는 한계

- **현상:** "수료율이 70%라면 교육비 지원을 받을 수 있나요?" 질문에서 FAISS는 정확한 근거 Chunk(`HR-EDU-004`)를 찾았지만, Qwen은 "네, 지원받을 수 있습니다"라고 오답
- **원인:** `HR-EDU-004`에는 "수료율 80% 미만이면 제외"라고 되어 있는데, Base Qwen이 "70%"와 "80% 미만"이라는 조건문을 정확히 대조하지 못함
- **확인:** 검색 단계(Retrieval)는 의도대로 동작했고, 문제는 생성 단계(Generation)의 조건 추론에서 발생했음을 두 단계를 분리해서 확인
- **개선 방향:** 이런 조건부 판단이 반복되는 질문 유형은, 검색만으로는 한계가 있으므로 SFT 학습 데이터에 유사한 조건문 판단 예시를 포함시켜 보완이 필요
- **배운 점:** 관련 Chunk가 검색됐다는 사실과, 그 내용을 LLM이 올바르게 추론한다는 것은 별개의 문제입니다. RAG와 SFT를 왜 함께 써야 하는지를 이론이 아니라 실패 사례로 체감했습니다.

4. 최종 학습 결론

평가 결과(`outputs/evaluation_summary.json`) 기준 20개 문항 중 Fine-tuned 모델이 18승(Base 1승, 1무)을 거뒀고, Keyword Score는 0.065 → 0.2, Token F1은 0.096 → 0.22로 개선되었습니다. 전체 파라미터의 1.18%만 학습했는데도 이 정도 응답 품질 차이가 났다는 점, 그리고 Base 모델의 점수 자체가 매우 낮았다는 점(Total Score 0.086)에서 범용 모델이 사내 고유 규정을 얼마나 모르는지를 수치로 체감할 수 있었습니다. 다만 Fine-tuned 모델의 Total Score도 0.285로 여전히 낮은 편이라, `SFT_EPOCHS`를 늘리거나 학습 데이터를 보강하면 더 개선될 여지가 있어 보입니다. 두 모델 모두 Unknown Policy 질문에 대한 Hallucination Rate가 0%였던 점은, `build_training_dataset.py`에서 "모른다"고 답하도록 일부러 넣어둔 학습 데이터가 의도대로 작동했다는 뜻이라 인상적이었습니다.

앞으로 기업용 sLLM을 설계할 때는 먼저 이해관계자의 Pain Point를 정의하고, 최신 사실과 근거는 RAG로 관리할지, 반복되는 응답 형식과 업무 행동은 SFT로 학습시킬지를 구분해야 한다고 생각합니다. 여기에 검색 실패와 Hallucination을 탐지할 평가 기준까지 포함해야 실제 업무에서 신뢰할 수 있는 AI 서비스가 될 것 같습니다.